

MatchPass - Guia de Validações entre Microsserviços

Onde validar, o que validar e quando usar validação síncrona ou assíncrona no fluxo de compra de ingressos.

1. Regra geral

Use validação síncrona quando o serviço precisa decidir imediatamente se aceita ou rejeita a requisição. Use eventos Kafka quando a ação pode acontecer depois, como reserva, pagamento, geração de ingresso e envio de e-mail.

Ponto crítico: o frontend não deve ser fonte confiável para preço, disponibilidade ou validade de IDs. O backend deve buscar dados oficiais nos serviços donos da informação.

2. Validação síncrona vs assíncrona

Tipo	Quando usar	Exemplos no MatchPass
Síncrona via proxy / client HTTP	Quando a resposta precisa ser imediata para aceitar ou recusar uma operação.	Validar eventId, sectorId, ticketType, preço oficial e userId antes de criar o pedido.
Assíncrona via Kafka	Quando o processo pode continuar em etapas, baseado em eventos.	Reserva de estoque, criação de sessão de pagamento, confirmação de pagamento, geração de ticket e envio de e-mail.

3. Onde validar por serviço

Serviço	O que validar	Como validar	Observação
order-service	userId existe; eventId existe; sectorId pertence ao evento; ticketType é válido; preço oficial; quantity > 0.	Síncrono via proxy para user-profile/identity e event-catalog.	É o ponto mais importante: não confiar no appliedPrice vindo do frontend.
inventory-service	eventId e sectorId válidos; inventário existe; quantidade disponível; reserva ainda pode ser criada.	Síncrono para validar evento/setor no event-catalog; validação local para estoque.	É dono do estoque, então deve validar disponibilidade de forma autônoma.
payment-service	orderId presente; userId presente; items não vazio; unitAmount > 0; quantity > 0; currency válida.	Validação local. Não precisa consultar event-catalog.	Deve receber valores já calculados pelo order-service.
ticket-service	OrderConfirmedEvent válido; items não vazio; reservationId presente; quantity > 0; não gerar duplicado.	Validação local e consulta ao próprio banco por orderId.	Deve ser idempotente para não gerar tickets duplicados em reprocessamentos Kafka.
notification-service	E-mail presente; template montado; items não vazio; qrCodeHash presente para anexos.	Validação local defensiva.	Não deve fazer regra de negócio pesada; só proteger contra eventos incompletos.

4. Validações no checkout

O checkout é o ponto mais sensível, porque vem diretamente de uma requisição do cliente. O request ideal deve conter apenas IDs, tipo de ingresso e quantidade. O preço deve ser obtido pelo backend.

Request recomendado:

Enviar: `userId`, `eventId`, `sectorId`, `ticketType`, `quantity`.

Não enviar: `appliedPrice` como valor confiável. Se o campo existir por enquanto, use apenas para teste, não como regra final.

Checklist do order-service

- Validar se o `userId` existe no `user-profile-service` ou `identity-service`.
- Validar se o `eventId` existe no `event-catalog-service`.
- Validar se o `sectorId` pertence ao `eventId`.
- Validar se o `ticketType` é permitido para aquele setor/evento.
- Buscar o preço oficial no `event-catalog-service`.
- Calcular `subtotal` e `totalAmount` no backend.
- Criar o pedido somente após essas validações.
- Publicar o evento de reserva somente se o pedido foi criado com dados oficiais.

5. Validações no inventory-service

- Validar `eventId` e `sectorId` com o `event-catalog-service` antes de reservar.
- Verificar se existe `EventSectorInventory` para `eventId` + `sectorId`.
- Verificar se `availableTicketsAmount` $\geq$ `quantity`.
- Criar `TicketReservation` com TTL configurado, por exemplo 30 minutos = 1800 segundos.
- Ao receber pagamento aprovado, confirmar venda e mover reservado para vendido.
- Ao receber pagamento falho ou expirado, liberar a reserva.

6. Validações no payment-service

- Validar se o `PaymentRequestedEvent` possui `orderId`, `userId`, `currency` e `items`.
- Validar se cada item possui `name`, `unitAmount` e `quantity`.
- Garantir que `unitAmount` esteja em centavos.
- Criar sessão Stripe com `line_items` separados.
- Configurar `expires_at` alinhado ao TTL da reserva, por exemplo 30 minutos.
- No webhook, transformar pagamento aprovado em `PaymentApprovedEvent` e expiração/falha em `PaymentFailedEvent`.

7. Validações no ticket-service

O ticket-service deve confiar no evento OrderConfirmedEvent como gatilho, mas precisa se proteger contra duplicidade e eventos incompletos.

- Validar se orderId, eventId e userId estão presentes.
- Validar se items não está nulo ou vazio.
- Validar se cada item possui orderItemId, sectorId, reservationId, ticketType e quantity > 0.
- Antes de gerar tickets, verificar se já existem tickets para o orderId.
- Se já existirem tickets para o orderId, ignorar o evento e registrar log.
- Gerar um ticket individual por unidade comprada.
- Gerar QR Code/hash para cada ticket.
- Publicar TicketGeneratedEvent somente após salvar os tickets.

8. Validações no notification-service

O notification-service deve ser tolerante a eventos incompletos. Ele não deve derrubar o fluxo inteiro por uma mensagem antiga ou malformada.

- Validar se customerEmail está presente nos eventos de pagamento.
- Validar se items não está nulo antes de chamar size() ou stream().
- Para tickets, validar se qrCodeHash está presente antes de gerar imagem PNG.
- Se o evento estiver incompleto, registrar log e ignorar, ou futuramente enviar para DLQ.
- Para envio dos ingressos por e-mail, anexar QR Codes em Base64.
- Substituir mocks de nome/e-mail por dados reais vindos do user-profile-service.

9. Ordem recomendada de implementação

Prioridade	Ação	Motivo
1	Remover confiança no appliedPrice do frontend.	Evita manipulação de preço pelo cliente.
2	Criar proxy do event-catalog no order-service.	Permite validar evento, setor, ticketType e preço oficial.
3	Criar proxy do user-profile/identity no order-service.	Permite validar usuário e obter nome/e-mail real para notificações.
4	Adicionar idempotência no ticket-service por orderId.	Evita tickets duplicados em reprocessamento Kafka.
5	Adicionar validações defensivas no notification-service.	Evita erro por eventos antigos ou incompletos.
6	Depois, configurar DLQ/retry para consumers Kafka.	Melhora resiliência em produção.

10. Resumo final

O fluxo principal de compra está fechado. A próxima fase é tornar o sistema confiável: cada serviço deve validar apenas aquilo que pertence à sua responsabilidade, usando chamadas síncronas quando precisa decidir imediatamente e eventos Kafka quando a ação pode ocorrer depois.